

EXERCICE : Le Moteur de Recommandation de Médias

Contexte : On modélise un réseau simplifié de divertissement et de partage de médias. On dispose d'un dictionnaire où chaque clé représente un utilisateur (identifié par son nom ou son ID) et chaque valeur associée est la liste des films ou séries qu'il a préférés. L'objectif est d'analyser la proximité structurelle entre les différents utilisateurs afin de leur suggérer du contenu affinitaire, puis de visualiser la structure globale de cette communauté sous forme graphique.

Partie 1 :

Pour mesurer mathématiquement la similarité entre deux profils d'utilisateurs A et B , on utilise couramment l'**indice de Jaccard**. Cet indicateur se définit par la taille de l'intersection de leurs ensembles de films regardés, divisée par la taille de leur union :

$$\text{Jaccard}(A, B) = \frac{|\text{Films}_A \cap \text{Films}_B|}{|\text{Films}_A \cup \text{Films}_B|}$$

Un étudiant écrit une fonction pour calculer cet indice en utilisant deux boucles `for` imbriquées. Pour vérifier si un film est commun aux deux profils, il utilise l'opérateur séquentiel `in` sur des listes Python classiques (`list`), par exemple : `if film in liste_B`.

1. **Question de réflexion :** Pourquoi l'opérateur `in` appliqué sur une structure de type `list` devient-il catastrophique en termes de performance (complexité algorithmique) lorsque la liste de films d'un utilisateur grandit ou contient des milliers d'entrées ? Expliquez ce qu'il se passe en arrière-plan au niveau de la mémoire.
2. **Le Défi Logique :** Quelle structure de données native de Python permet de calculer des intersections et des unions de manière quasi instantanée ? Réécrivez la formule de l'indice de Jaccard ci-dessus en utilisant la syntaxe et les opérateurs exacts de cette structure Python (sans appeler de boucles explicites).

Partie 2 :

On souhaite représenter visuellement la matrice de similarité calculée pour un échantillon de 100 utilisateurs. Utilisez la fonction de cartographie bidimensionnelle `plt.imshow()` de la bibliothèque `matplotlib` pour afficher une grille de pixels (une *heatmap* / carte thermique).

Dans ce graphique, les axes X et Y représentent les indices des utilisateurs (de 0 à 99). La couleur d'un pixel situé aux coordonnées (i, j) traduit l'intensité du score de Jaccard entre l'utilisateur i et l'utilisateur j (le bleu foncé correspond à un score de 0 et le rouge vif correspond à un score maximal de 1).

En affichant le graphique final à l'écran, Vous constatez deux propriétés géométriques marquantes :

- Une ligne diagonale continue traverse tout le graphique du coin supérieur gauche au coin inférieur droit, et elle est intégralement colorée en rouge vif (valeur de 1.0).
- Le graphique présente une symétrie bilatérale parfaite de part et d'autre de cette diagonale principale.

3. **Question** : Expliquez logiquement et mathématiquement pourquoi ces deux phénomènes visuels (la diagonale rouge constante et la symétrie parfaite) apparaissent obligatoirement, quelles que soient les données de consommation initiales des utilisateurs. Que signifierait concrètement la présence d'un pixel rouge vif situé en dehors de la diagonale ?

Partie 3 :

4. Écrivez une fonction Python nommée `extraire_reseau_fort(similarites, seuil)` qui répond au cahier des charges suivant :
- Elle reçoit en paramètre la matrice bidimensionnelle NumPy des scores de similarité ainsi qu'une valeur numérique flottante appelée seuil (comprise entre 0 et 1).
 - Elle applique un filtre logique strict sur la matrice : si la similarité entre deux utilisateurs distincts est strictement inférieure au seuil, le score est ramené à 0.
 - Elle configure et affiche le graphique final à l'aide de `plt.imshow()`.
 - **Contrainte absolue** : Si, après application du filtre, la matrice résultante ne contient plus que des valeurs nulles (en ignorant la diagonale principale), la fonction ne doit pas afficher un graphique vide mais doit lever une exception ou afficher un message d'alerte textuel explicite à l'écran.